

OAK Community

From an architecture brief to a signed, independently verified plan — entirely on your own machine.

What this is. OAK Community 0.7.1 is a *local-first developer release*. It carries no production or customer readiness claim, and no external security review was commissioned for it. Everything in this manual runs on your own machine against public or synthetic fixtures; the only mutation the runner can ever perform is creating and removing one network-isolated, never-started container that you explicitly acknowledge. The authoritative statements of scope and limitation are `SECURITY.md` and `docs/security/residual-risk.md` in the repository.

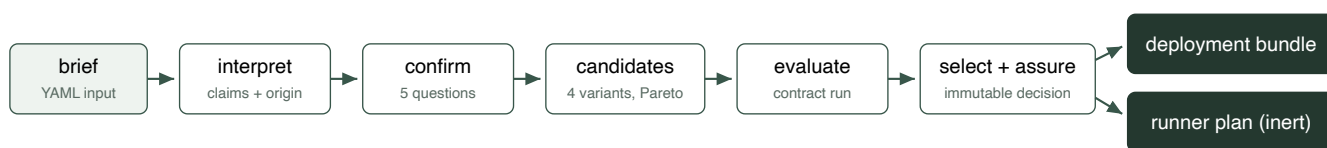
Covers OAK Community 0.7.1 · generated from `docs/manual/` in the repository — see the colophon for how to rebuild this document and re-capture its screenshots from a live system.

CONTENTS

1. What OAK Community is — and is not
2. Installing on a fresh machine
3. The command-line journey: brief to compiled bundle
4. The browser workspace
5. The signed runner: sign, approve, dispatch, verify, revoke
6. Verifying release artifacts
7. Troubleshooting
8. Uninstalling completely
9. Colophon: how this manual is built

What OAK Community is — and is not

OAK Community is a compiler-shaped control plane for designing, evaluating and planning AI systems. You feed it an **architecture brief** — a bounded YAML description of a problem, its stakeholders, data, quality priorities and constraints — and it walks that brief through a deterministic pipeline: it interprets the brief into typed claims with provenance, asks you at most five ranked clarification questions, generates and compares candidate architectures against hard constraints, evaluates the selected candidate against an evaluation contract, records an immutable decision with your rationale, attaches an assurance plan, and finally compiles a canonical **deployment bundle** and a typed, non-executing **runner plan**.



Every stage is deterministic, offline, and writes immutable content-addressed artifacts with an audit trail.

Figure 1 — the pipeline. The compiled plan is inert: executing anything requires the separate signing, approval and runner verification described in chapter 5.

Three properties shape everything you will see. **Deterministic:** the same inputs produce byte-identical canonical artifacts, so digests are meaningful and two clean machines agree. **Offline:** after installation, no command in this manual needs a network, a hosted service, or a model provider. **Fail-closed:** anything unknown, malformed, expired, unsigned or mismatched is refused with a stable `OAK-*` error code, never skipped.

What it is not

It is not a deployment tool for real infrastructure: the only target it will ever touch is an explicitly acknowledged, network-isolated local fixture. It is not multi-tenant, has no authentication, and binds to loopback. It is not a hosted service and never phones home. And a compiled plan is not permission to run anything — execution authority comes only from the signing and approval chain in chapter 5, verified independently by the runner.

Installing on a fresh machine

Prerequisites

REQUIREMENT	VERSION	NOTES
macOS or Linux	macOS 14+ arm64/x86_64, Linux glibc $\geq$ 2.24 (x86_64) / $\geq$ 2.27 (aarch64)	See <code>docs/platforms.md</code> for the authoritative matrix
<code>git</code>	any recent	Required even offline: the hygiene check shells out to it
<code>uv</code>	0.10.x	Installs and pins Python 3.13.12 itself
Node.js + <code>pnpm</code>	<code>pnpm</code> 11.15.1	Only for the web workspace. <code>pnpm</code> downloads and runs the pinned Node 24.18.0 itself, so your system Node version does not matter
Docker with Compose	any recent	Only for the web stack (ch. 4) and the runner's apply cycle (ch. 5)

Install

```
git clone https://github.com/nmasamba/OAK.git
cd OAK
make bootstrap
```

`make bootstrap` installs the locked Python and web dependencies and downloads `pnpm`'s managed Node runtime. It is the one step that uses the network; everything after it works with egress disabled.

```
uv run oak --version
0.7.1
```

Throughout this manual, commands are shown as `uv run oak ...` so they work from the repository without activating anything. If you prefer, activate the environment once (`source .venv/bin/activate`) and drop the `uv run` prefix.

Optional: run the full verification gate

```
make check
```

This runs formatting, linting, type checks, boundary rules, contract validation and the unit, integration and end-to-end suites — around ten minutes. The PostgreSQL-gated suites skip silently unless a

database is published; that is expected on a fresh machine and documented in `docs/development.md`.

The command-line journey: brief to compiled bundle

This chapter follows the reference brief that ships with the repository — a citation-first documentation-search service — from intake to a compiled bundle. Every command runs offline. Start in the repository root and create a workspace:

```
uv run oak init ./workspace
cd workspace
```

1 • Ingest and interpret the brief

```
uv run oak design ../examples/briefs/public-manual-qa.yaml
```

OAK quarantines the brief as an untrusted source record, then deterministically interprets it into a typed intent. Every populated value carries provenance: a *fact* quoted from the brief, an *inference*, or a *domain default* — you will see these classes again in the browser workspace.

2 • Review the questions, then confirm

```
uv run oak questions
```

At most five ranked questions, each explaining why it matters to candidate choice. Answers live in a small YAML file; the reference answers ship alongside the brief:

```
uv run oak confirm --answers ../examples/briefs/public-manual-qa-answers.yaml
```

Confirmation creates immutable successors for the affected claims — the original interpretation is never edited, and the audit trail records who decided what.

3 • Generate and compare candidates

```
uv run oak candidates
```

Four provider-neutral candidate architectures are generated from a bundled synthetic catalogue, checked against hard constraints, and scored with transparent estimator ranges (cost, latency, quality, operability, energy). Infeasible candidates stay visible with their rejection reasons — in the reference case, `candidate-04` fails a hardware constraint and says so.

4 • Evaluate, select, assure

```
uv run oak evaluate candidate-03
printf 'Balanced trade-off between quality, cost, and operability.\n' > decision.md
uv run oak select candidate-03 --rationale-file decision.md
uv run oak assure candidate-03 --output ./assurance
```

`evaluate` runs the deterministic reference evaluation contract and records pass/fail per metric.

`select` records an immutable decision bound to you (`OAK_ACTOR`), your rationale, and the rejected alternatives. `assure` writes the assurance plan: required tests, evidence, controls, owners, and the gate blockers that stand between this design and any real deployment.

5 • Compile the bundle and the runner plan

```
uv run oak plan candidate-03 \
  --target ../examples/targets/local-fixture.yaml \
  --output ./bundle
```

The target profile is explicit invocation data — OAK never treats your machine as the target. Compilation validates the profile, binds it to the tenant, checks the candidate's platform and capacity requirements against it, and emits the canonical files: the semantic manifest, the deployment bundle, and a `draft` typed runner plan whose verification policy is derived from that exact target. For this read-only fixture the plan contains only `inventory`, `validate`, `render`, `plan` and `verify` operations.

Two clean machines running this chapter produce byte-identical semantic manifests — that is the determinism promise, and it is what makes the digests in chapter 5 worth signing.

Export and import

```
uv run oak export --output ../case-export
uv run oak import ../case-export --workspace ../restored
```

The export is the portable, digest-verified unit — also your backup before any upgrade. Import revalidates every artifact, digest and audit event before publishing the restored workspace.

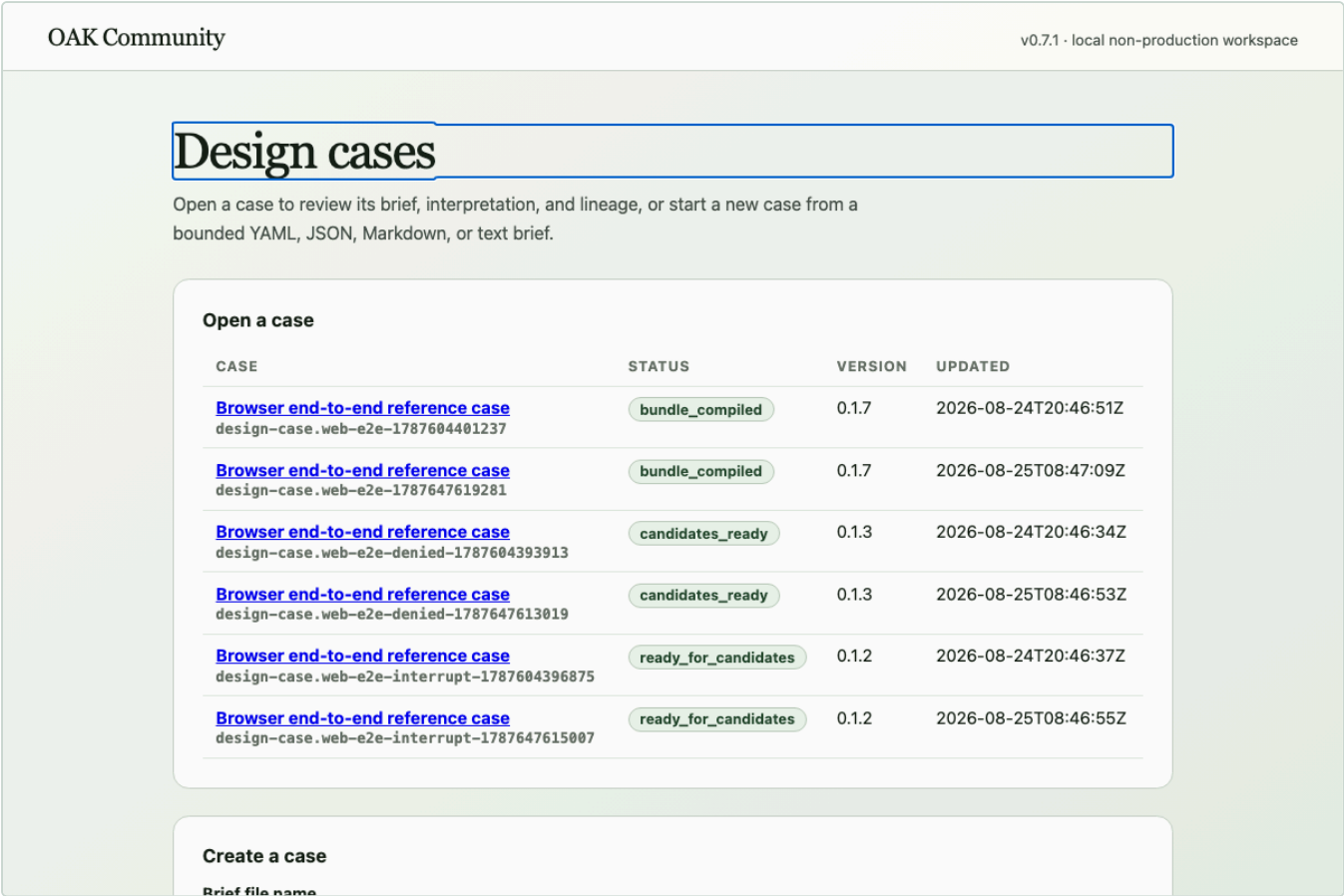
The browser workspace

The web workspace drives the same journey through a reviewer-friendly interface, backed by a persistent PostgreSQL control plane. Start the stack:

```
docker compose up -d --build
```

Always pass `--build`: Compose happily reuses a stale cached image after you change source. Confirm what you are running with `curl http://127.0.0.1:8080/version` — it must report 0.7.1.

The API listens on `127.0.0.1:8080` and the workspace on `http://127.0.0.1:5173` — loopback only, by design. The screenshots below are captured from a real journey against this stack.



The case list. Paste a brief (or start from the reference brief) and create a case.

Browser end-to-end reference case design-case.web-e2e-denied-1787647613019	candidates_ready	0.1.3	2026-08-25T08:46:53Z
Browser end-to-end reference case design-case.web-e2e-interrupt-1787604396875	ready_for_candidates	0.1.2	2026-08-24T20:46:37Z
Browser end-to-end reference case design-case.web-e2e-interrupt-1787647615007	ready_for_candidates	0.1.2	2026-08-25T08:46:55Z

Create a case

Brief file name

getting-started-1787656972840.yaml

Brief content

```
network: isolated-no-egress-after-setup
hardware:
  cpu_architectures: [x86_64]
  ram_gib: 32
  storage_gib: 100
openness:
  software: OSI-approved by default
  models: local optional model or no-model baseline
unknowns:
  - Exact document count and update cadence for a real deployment.
  - Approved model licence and hardware if the generated-answer variant is selected.
extensions: {}
```

Create case

This workspace has no target mutation, secret resolution, or inference-traffic path. Compiled plans stay draft review artifacts.

Creating a case: the brief file name and content, exactly as the CLI would ingest them.

DESIGN CASE

Browser end-to-end reference case

design-case.getting-started-1787656972840

Current state

Status

needs_confirmation

Version

0.1.1

Updated

2026-08-25T11:22:53Z

Open questions

5

[Refresh](#)

This case needs its open questions answered. [Answer the 5 ranked questions](#) after [reviewing the interpreted claims](#).

[Review the brief interpretation](#) to see which values are facts, inferences, defaults, or unknowns.

Audit timeline[Download the canonical export](#)1. [case_created](#) case version 0.1.0

#1 · local-user via api · 2026-08-25T11:22:53Z

After “Interpret brief” the case reaches *needs_confirmation*; the next actions are server-driven, so the UI can never invent a transition the control plane would refuse.

BRIEF AND INFERENCE REVIEW

Browser end-to-end reference case

design-case.getting-started-1787656972840 · intent 0.1.0 · draft

Interpreted claims and their origin

Every value below is labelled with how it entered the design: stated in the brief, inferred, supplied as a domain default, corrected by a reviewer, or still unknown. Inferred and default values deserve scrutiny before candidates are trusted.

FIELD	VALUE	ORIGIN	MATERIALITY	RATIONALE
/spec/data/classifications/0	public	Fact from brief	high confidence 1.00	Copied from brief field data.classifications.
/spec/data/extensions/oak.community-1production_data_permitted	false	Fact from brief	critical confidence 1.00	Copied from the brief production-data declaration.
/spec/data/sources/0	public technical manuals	Fact from brief	high confidence 1.00	Copied from brief field data.sources.
/spec/decision/actions/0	return cited passages	Fact from brief	high confidence 1.00	Copied from brief field decision.actions.
/spec/decision/actions/1	draft a cited answer	Fact from brief	high confidence 1.00	Copied from brief field decision.actions.
/spec/decision/act	abstain	Fact from brief	high	Copied from brief field

Claim review: every value is labelled a fact from the brief, an inference, or a domain default, with the unknowns listed beneath.

QUESTION REVIEW

Browser end-to-end reference case

design-case.getting-started-1787656972840 · case version 0.1.1

5 ranked questions block this design

Decisions are recorded with your local actor identity and a value digest. You can answer a subset now; unanswered questions stay open and the case remains awaiting confirmation.

Which approved model licence and measured target hardware apply? critical

1. question.model-hardware · affects /spec/hardware · blocks solve

Model eligibility and measured capacity can make generated-answer variants infeasible.

- ☐ Confirm the current value
- ☐ Correct it to a different value
- ☐ Reject the claim
- ☐ Accept the risk of leaving it open

Confirm whether production data is permitted for this design case. critical

2. question.production-use · affects /spec/data/extensions/oak.community-1production_data_permitted · blocks assess

This changes the data boundary, controls, and deployment eligibility.

- ☐ Confirm the current value
- ☐ Correct it to a different value

The five ranked questions. Confirm, correct (with a value), reject, or accept-risk — each with a recorded rationale.

DURABLE OPERATION

generate candidates

operation.2ed7b803e939c1cfb741c57eb878cc5da19dd02104408913dfce31009d11d356

Progress

State

queued

Attempts

0 of 3

Created

2026-08-25T11:22:54Z

Completed

not yet

Waiting for the worker to finish...

Cancel operation

[Back to case design-case.getting-started-1787656972840](#)

This workspace has no target mutation, secret resolution, or inference-traffic path. Compiled plans stay draft review artifacts.

Long stages run as durable operations: they survive restarts, can be cancelled cooperatively, and record their outcome.

CANDIDATE COMPARISON

Browser end-to-end reference case

design-case.getting-started-1787656972840 · case version 0.1.3 · candidates_ready

4 candidate architectures

The simpler baseline appears first. Objective values are transparent estimator outputs with ranges; every raw document is inspectable below. Infeasible variants stay visible with their rejection reasons.

CANDIDATE	VARIANT	STATUS	FRONTIER	MONTHLY COST	LATENCY P95	QUALITY
candidate-00	simpler_baseline	feasible	member	8 USD/month (7.2–8.8)	180 milliseconds (135–225)	0.62 normalized_ (0.57–0.67)
candidate-01	minimum_sufficient	feasible	member	8 USD/month (7.2–8.8)	180 milliseconds (135–225)	0.62 normalized_ (0.57–0.67)
candidate-03	balanced_enterprise	feasible	member	36 USD/month (32.4–39.6)	1,200 milliseconds (900–1,500)	0.86 normalized_ (0.81–0.91)
candidate-04	high_assurance_sovereign	infeasible constraint.hardware: Required accelerator compatibility is unconfirmed.	—	82 USD/month (73.8–90.2)	650 milliseconds (487.5–812.5)	0.93 normalized_ (0.88–0.98)

Candidate comparison. Note the infeasible candidate staying visible with its constraint failure — rejection reasons are first-class.

DECISION AND ASSURANCE

Browser end-to-end reference case

design-case.getting-started-1787656972840 · case version 0.1.6 · assurance_planned

Selection decision

Selected candidate: **candidate-03**

Balanced trade-off between quality, cost, and operability.

Decided by local-user at 2026-08-25T11:22:59Z

Alternatives and why they were not selected

- **candidate-00** — Retained as a visible feasible trade-off but not selected by the owner.
- **candidate-01** — Retained as a visible feasible trade-off but not selected by the owner.
- **candidate-04** — constraint.hardware: Required accelerator compatibility is unconfirmed.

► Raw decision document**Evaluation evidence**

Contract result: **pass** · fixture public-manual-qa-evaluation-1.0.0

The immutable decision with evaluation evidence, plus the assurance plan's controls and gate blockers.

BUNDLE REVIEW

Browser end-to-end reference case

design-case.getting-started-1787656972840 · case version 0.1.7 · bundle_compiled

Plan, approval, and apply

Plan

draft

compiled, unsigned

Approval

none recorded — requires environment-owner and security-reviewer

Apply

unavailable in Community — no runner execution authority

This bundle is not authorized for target execution on its own: dispatch requires a separately signed plan signature, current approvals, and independent runner verification.

Component lock

COMPONENT	VERSION	LICENCE	SOURCE
component.fixture-lexical-search sha256:833ca7370b4faa034e25b183f4340741d4ada7477fbbb3424ee8bfebc93a31b	1.0.0	allowed	Digest-verified Community catalogue snapshot
component.fixture-local-model sha256:cf13689ae4ad43dfeebe008e456d27494f07eb0472a440cf0890a78641635152	1.0.0	allowed	Digest-verified Community catalogue snapshot

Bundle review. Plan, approval and apply are deliberately separated — and apply is marked unavailable here: the web surface holds no runner execution authority.

DESIGN CASE

Browser end-to-end reference case

design-case.getting-started-1787656972848

Current state

Status

bundle_compiled

Version

0.1.7

Updated

2026-08-25T11:23:01Z

Open questions

0

[Refresh](#)[Review the brief interpretation](#) to see which values are facts, inferences, defaults, or unknowns.**Compiled bundle**[Review the compiled bundle](#) — normalized review files, component lock, and the explicit plan/approval/apply separation.**Audit timeline**

The audit timeline: every command, actor and artifact from case creation to *bundle_compiled*, exportable as a canonical JSON bundle.

Stop the stack with `docker compose down` (add `--volumes` to also drop the local database and artifact store).

The signed runner: sign, approve, dispatch, verify, revoke

Everything so far produced *inert* artifacts. This chapter is where OAK lets something run — under the strictest discipline in the system. Two separate trust domains cooperate without ever trusting each other's say-so:

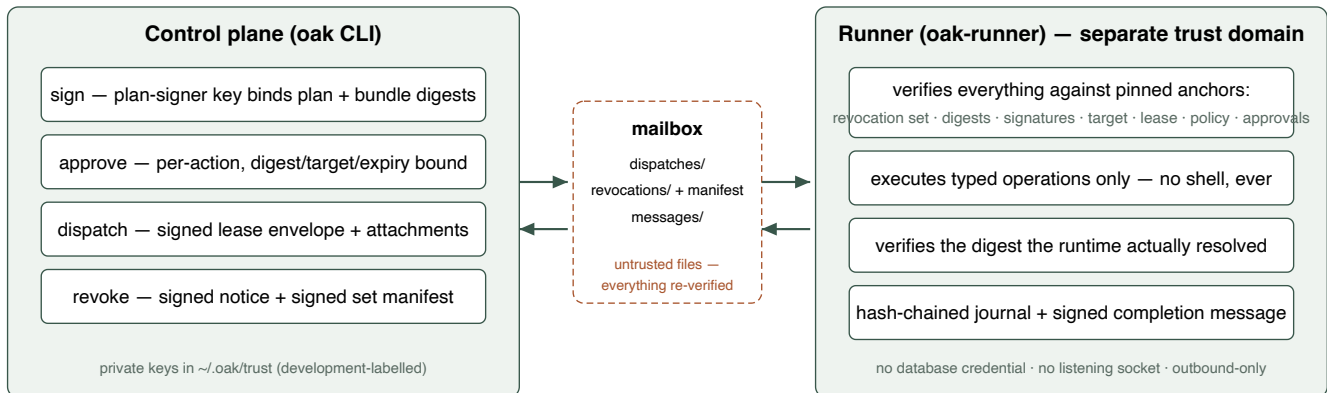


Figure 2 — two trust domains, one untrusted mailbox. A tampered mailbox file is an expected adversarial case: the runner recomputes and re-verifies everything from pinned keys, and any failure is a denial with a stable code, published back as a signed message.

Set up and compile against the mutation fixture

The runner demonstration uses the one mutation-capable target that ships: an acknowledged, network-isolated fixture whose only permitted mutation is creating and removing a single never-started container (Docker required). Set the environment, then compile chapter 3's journey against that target:

```

export OAK_TRUST_DIRECTORY=$PWD/trust
export OAK_DISPATCH_MAILBOX=$PWD/mailbox
export OAK_ACTOR=local-user

uv run oak init ./runner-demo && cd runner-demo
uv run oak design ../examples/briefs/public-manual-qa.yaml
uv run oak confirm --answers ../examples/briefs/public-manual-qa-answers.yaml
uv run oak candidates
uv run oak evaluate candidate-03
printf 'Balanced fixture for the runner demonstration.\n' > decision.md
uv run oak select candidate-03 --rationale-file decision.md
uv run oak assure candidate-03 --output ./assurance
uv run oak plan candidate-03 \
  --target ../examples/targets/local-mutation-fixture.yaml \
  --output ./bundle
  
```

Sign, approve, dispatch

```
uv run oak keys init
uv run oak sign
uv run oak approve dry_run
uv run oak approve apply
uv run oak approve rollback
uv run oak dispatch apply verify rollback
```

`keys init` creates three development-labelled Ed25519 identities with separated roles — the plan-signer may not approve, and the runner enforces that the two signatures come from different keys. Each `approve` is bound to one action, the exact plan and bundle digests, the target's identity and fingerprint, a nonce, and an expiry. `dispatch` writes the signed lease envelope and every attachment into the mailbox — delivery, not execution.

Run the runner

```
OAK_RUNNER_MAILBOX=$OAK_DISPATCH_MAILBOX \
OAK_RUNNER_TRUST_ANCHORS=$OAK_TRUST_DIRECTORY \
OAK_RUNNER_TARGET_PROFILE=../examples/targets/local-mutation-fixture.yaml \
OAK_RUNNER_HOME=$PWD/runner-home \
uv run oak-runner run-once
```

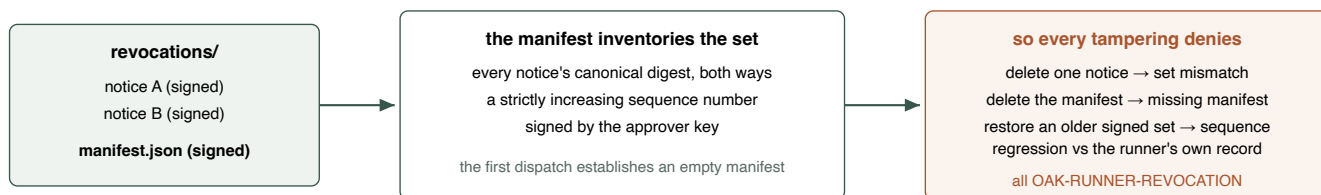
The runner independently verifies the revocation set, the envelope, every digest and signature against its own pinned anchors, the target fingerprint it recomputes locally, the lease and nonce, the compiled verification policy's clauses, the adapter allowlist and typed parameters, and the approvals — all before touching anything. It then creates the fixture container with `docker create --network=none`, reads back the image digest the daemon actually resolved and refuses on any mismatch, verifies, rolls back — removing exactly what it created — and publishes a signed completion message. Then:

```
uv run oak ingest --output json
uv run oak-runner status
```

`ingest` verifies the runner's signed messages — delivery never implies success — and `status` reports the journal's hash-chain integrity and any manual-recovery state.

Revocation — and why deleting files does not help

```
uv run oak revoke-approval apply --reason "manual drill"
```



A revoked approval stays revoked: the runner would rather deny every pending dispatch than guess about a revocation set it cannot prove.

Figure 3 — the revocation channel. Notices and their signed set manifest; the runner records the highest manifest sequence it has seen in its own home directory.

Verifying release artifacts

If you received OAK Community as built artifacts rather than as this repository, verify them before installing anything. The repository builds with:

```
make release
```

which builds the wheel and sdist twice and fails if the digests differ, installs the wheel into a clean environment holding only the locked runtime closure, runs it from outside the checkout, and emits an SBOM, a licence inventory, and `SHA256SUMS`. On the consuming side:

```
make verify-release
```

is a dependency-free verifier for the `SHA256SUMS` manifest. Be aware of what this does and does not prove: checksums prove the bytes match the manifest; they do not prove who produced them. Release artifacts are deliberately not signed yet (`RR-005` in the residual-risk register), and the container images are not byte-reproducible (`RR-006`) — their SBOMs and provenance under `docs/release/0.7.1/` record what one build produced.

Troubleshooting

SYMPTOM	LIKELY CAUSE	WHAT TO DO
A command refuses with <code>OAK-EXPECTED-VERSION</code>	The case advanced since you last read it	Re-read the case and retry with the current version — this is optimistic concurrency working, not an error to force past
The web stack serves an old version	<code>docker compose up -d</code> reused a cached image	Always <code>--build</code> , then check <code>/version</code>
<code>OAK-RUNNER-REVOCATION</code> on every dispatch	The revocation manifest is missing or the notice set does not match it	Re-publish from the control plane (<code>oak revoke-approval</code> , or a fresh dispatch establishes an empty manifest). Do not hand-delete notices — the mismatch is the protection working
<code>OAK-RUNNER-REPLAY</code> on every dispatch	The consumed-nonce ledger in <code>\$OAK_RUNNER_HOME</code> is corrupt; it deliberately refuses rather than reading as empty	Inspect <code>consumed-nonces.json</code> ; remove it only if you accept that previously burned nonces become replayable
<code>OAK-WORKSPACE-CORRUPT</code> on every command	Manifest unreadable, or written by a format this build does not know	Restore from your last <code>oak export</code> — this is why the manual says export before upgrading
Runner exits 64 / 70 with a code on stderr	64 is configuration refusal, 70 is a verification or execution denial	Look the code up in <code>docs/error-codes.md</code> — every <code>OAK-*</code> code is documented with its raise site

The full runbook is `docs/operations.md`; the complete generated error-code reference is `docs/error-codes.md`.

Uninstalling completely

```
docker compose down --volumes      # stack, database and artifact volumes
rm -rf ~/.oak                      # trust keys, runner home, local state
cd .. && rm -rf OAK                 # the repository, workspaces and venv
```

`~/.oak` holds private signing keys, the runner's journal, consumed nonces and its revocation-sequence mark — export anything you want to keep first. Shared caches (`~/.cache/uv`, the pnpm store, Playwright browsers) and the third-party images OAK pulled (`postgres:17.6-alpine`, `python:3.13.12-slim`, `node:24.18.0-alpine`, `nginxinc/nginx-unprivileged:1.29.1-alpine`) are yours to keep or remove with `docker image rm`. `scripts/check_clean_machine.py` reports what OAK-related state remains.

Colophon: how this manual is built

The source of this manual is `docs/manual/manual.html`; the PDF is a rendering of it. Both are committed. To rebuild:

```
# screenshots, from a live journey against the Compose stack
docker compose up -d --build
OAK_MANUAL_SCREENSHOTS=1 pnpm --dir web exec playwright test e2e/manual-screens.spec.ts

# the PDF, using the same pinned Chromium the e2e suite uses
node docs/manual/build_manual.mjs
```

Every command in chapters 3 and 5 is the exact invocation exercised by the repository's end-to-end suites (`tests/e2e/test_cli.py`, `tests/e2e/test_runner_journey.py`), so the suite passing means this manual's commands work. The diagrams are inline SVG in the source. This manual describes OAK Community 0.7.1 — a local-first developer release with no production or customer readiness claim; where this document and `docs/security/residual-risk.md` could ever disagree, the register wins.